

Experiment 4

Student Name: Nikhil Kumar

UID: 25MCI10036

Branch: MCA (AI/ML)

Section/Group: 25MAM-1(A)

Semester: 2-SEM

Date of Performance: 03/02/2026

Subject Name: Technical Training - 1

Subject Code: 25CAP-652

1. Aim of the Session:

To understand and implement iterative control structures in PostgreSQL conceptually, including FOR loops, WHILE loops, and basic LOOP constructs, for repeated execution of database logic.

2. Software Requirements:

- PostgreSQL
- pgAdmin
- Windows Operating System

3. Objectives of the Session:

- To understand why iteration is required in database programming
- To learn the purpose and behaviour of FOR, WHILE, and LOOP constructs
- To understand how repeated data processing is handled in databases
- To relate loop concepts to real-world batch processing scenarios
- To strengthen conceptual knowledge of procedural SQL used in enterprise systems

4. Theory:

In real-world database applications, tasks often need to be repeated multiple times. Examples include processing employee records, generating reports, validating data, applying salary increments, and running batch jobs. Standard SQL is declarative and works well for single operations, but repeated logic requires procedural control.

PostgreSQL provides **PL/pgSQL**, a procedural extension that supports iteration using loop structures. These loops allow SQL statements to execute repeatedly until a specific condition is met.

Iteration in PostgreSQL is commonly used inside:

- Stored procedures
- Functions
- Anonymous execution blocks

Large organizations such as Amazon, SAP, Oracle, and Rippling use loop-based logic for payroll processing, billing cycles, analytics, and automation workflows.

Types of loops in PostgreSQL

1. FOR Loop (Range-Based)

- Executes a fixed number of times
- Useful when the number of iterations is known in advance
- Commonly used for counters, testing, and batch execution

2. FOR Loop (Query-Based)

- Iterates over rows returned by a query
- Processes one row at a time
- Frequently used for reporting, audits, and row-wise calculations

3. WHILE Loop

- Executes repeatedly as long as a condition remains true
- Suitable for condition-controlled execution
- Often used in retry logic or threshold-based processing

4. LOOP with EXIT Condition

- Executes indefinitely until explicitly stopped
- Provides maximum control over execution flow
- Used in complex workflows where exit conditions are custom-defined

5. Practical / Experiment Steps:

Example 1: FOR Loop – Simple Iteration

- The loop runs a fixed number of times
- Each iteration represents one execution cycle
- Useful for understanding basic loop behaviour

Application: Counters, repeated tasks, batch execution

Code

```
DO $$  
DECLARE  
    i INT;  
BEGIN  
    FOR i IN 1..5 LOOP  
        RAISE NOTICE 'Iteration number: %', i;  
    END LOOP;  
END $$;
```

Output

```
NOTICE: Iteration number: 1
NOTICE: Iteration number: 2
NOTICE: Iteration number: 3
NOTICE: Iteration number: 4
NOTICE: Iteration number: 5
DO

Query returned successfully in 129 msec.
```

Example 2: FOR Loop with Query (Row-by-Row Processing)

- The loop processes database records one at a time
- Each iteration handles a single row
- Simulates cursor-based processing

Application: Employee reports, audits, data verification

Code

```
create table employees(
emp_id int,
emp_name text,
salary numeric(10,2)
);
```

```
INSERT INTO employees VALUES
(1, 'Amit', 42000),
(2, 'Neha', 55000),
(3, 'Rahul', 48000),
(4, 'Priya', 62000);
```

```
DO $$
DECLARE
    rec RECORD;
BEGIN
    FOR rec IN SELECT emp_id, emp_name FROM employees LOOP
        RAISE NOTICE 'Employee ID: %, Name: %', rec.emp_id, rec.emp_name;
    END LOOP;
END $$;
```

Output

```
NOTICE: Employee ID: 1, Name: Amit
NOTICE: Employee ID: 2, Name: Neha
NOTICE: Employee ID: 3, Name: Rahul
NOTICE: Employee ID: 4, Name: Priya
DO

Query returned successfully in 108 msec.
```

Example 3: WHILE Loop – Conditional Iteration

- The loop runs until a condition becomes false
- Execution depends entirely on the condition
- The condition is checked before every iteration

Application: Retry mechanisms, validation loops

Code

```
DO $$
DECLARE
    counter INT := 1;
BEGIN
    WHILE counter <= 5 LOOP
        RAISE NOTICE 'Counter value: %', counter;
        counter := counter + 1;
    END LOOP;
END $$;
```

Output

```
NOTICE: Counter value: 1
NOTICE: Counter value: 2
NOTICE: Counter value: 3
NOTICE: Counter value: 4
NOTICE: Counter value: 5
DO

Query returned successfully in 125 msec.
```

Example 4: LOOP with EXIT WHEN

- The loop does not stop automatically
- An explicit exit condition controls termination
- Gives flexibility in complex logic

Application: Workflow engines, complex decision cycles

Code

```
DO $$
DECLARE
    x INT := 1;
BEGIN
    LOOP
        RAISE NOTICE 'Value: %', x;
        x := x + 1;

        EXIT WHEN x > 5;
    END LOOP;
END $$;
```

Output

```
NOTICE: Value: 1
NOTICE: Value: 2
NOTICE: Value: 3
NOTICE: Value: 4
NOTICE: Value: 5
DO

Query returned successfully in 106 msec.
```

Example 5: Salary Increment Using FOR Loop

- Employee records are processed one by one
- Salary values are updated iteratively
- Represents real-world payroll processing

Application: Payroll systems, bulk updates

Code

```
DO $$
DECLARE
    rec RECORD;
BEGIN
    FOR rec IN SELECT emp_id FROM employees LOOP
```

```
UPDATE employees  
SET salary = salary + 1000  
WHERE emp_id = rec.emp_id;  
END LOOP;  
END $$;
```

```
SELECT * FROM employees;
```

Output

	emp_id integer	emp_name text	salary numeric (10,2)
1	1	Amit	43000.00
2	2	Neha	56000.00
3	3	Rahul	49000.00
4	4	Priya	63000.00

Example 6: Combining LOOP with IF Condition

- Loop processes each record
- Conditional logic classifies data during iteration
- Demonstrates decision-making inside loops

Application: Employee grading, alerts, categorization logic

Code

```
DO $$  
DECLARE  
    rec RECORD;  
BEGIN  
    FOR rec IN SELECT emp_name, salary FROM employees LOOP  
        IF rec.salary >= 50000 THEN  
            RAISE NOTICE '% is a High Salary Employee', rec.emp_name;  
        ELSE  
            RAISE NOTICE '% is a Low Salary Employee', rec.emp_name;  
        END IF;  
    END LOOP;  
END $$;
```

Output

```
NOTICE: Amit is a Low Salary Employee  
NOTICE: Neha is a High Salary Employee  
NOTICE: Rahul is a Low Salary Employee  
NOTICE: Priya is a High Salary Employee  
DO
```

```
Query returned successfully in 146 msec.
```

6. Learning Outcomes (What I have learned):

This experiment helps students understand how iterative control structures work in PostgreSQL at a conceptual level. Students learn where and why loops are used in database systems and gain foundational knowledge required for writing procedural logic in enterprise-grade applications.